파이썬 완전 백과사전 2권 · 딥러닝 기초 교안

1일차

숫자를 담는 그릇과 기울기로 고쳐 가는 학습

1~5차시 · 책 4장 텐서 ~ 5장 자동 미분과 경사하강법

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/04-tensors, 05-autograd

과정 전체: 25차시, 3일 또는 5일

일	차시	주제	책 범위
1일	1~5	텐서, 자동 미분과 경사하강법	4장, 5장
2일	6~10	nn.Module로 첫 신경망, 데이터 파이프라인	6장, 7.1~7.6
3일	11~15	파이프라인 마무리, 잘 학습시키기	7.7~7.11, 8장
4일	16~20	합성곱 신경망(CNN)	9장
5일	21~25	전이학습: 사전학습 모델 빌려 쓰기	10장

3일 과정 = 1~3일, 15차시. 텐서에서 시작해 신경망을 잘 학습시키는 법까지

5일 과정 = 3일 과정 + 4~5일, 25차시. 이미지를 보는 CNN과 전이학습까지

한 차시 50분 · 하루 5차시 · 교재: 파이썬 완전 백과사전 2권 4~10장 · 예제: examples/vol2

오늘의 시간표

차시	주제	책 범위
1	텐서 만들기, 모양, 자료형, 인덱싱	4장 들어가며 ~ 4.4
2	모양 바꾸기, 브로드캐스팅, 장치, 기후 실습	4.5 ~ 4.11
3	기울기와 손으로 하는 경사하강법	5장 들어가며 ~ 5.3
4	autograd, 계산 그래프, 연쇄 법칙	5.4 ~ 5.7
5	기록 끄기, 학습률, 미니배치, 당뇨병 실습	5.8 ~ 5.13

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분

준비: examples/vol2 폴더에서 uv sync · 4·5장 예제 63개는 하나에 몇 초 안쪽, 내려받을 데이터가 없다

1차시

01

텐서: 숫자를 담는 그릇

책 4장 들어가며 ~ 4.4절

학습 목표

- 리스트 대신 텐서를 쓰는 이유를 말한다
- 텐서를 만들고 모양(shape)과 자료형(dtype)을 읽는다
- 인덱싱으로 꺼내고, 창(view)과 복사를 가른다

4장 사진 한 장은 숫자 몇 개일까

```
# 사진 한 장 크기의 숫자: 세로 1080 × 가로 1920 × 색 3개
리스트_사진 = [0.5] * (1080 * 1920 * 3)
텐서_사진 = torch.full((1080, 1920, 3), 0.5)

밝게 = [값 * 1.2 for 값 in 리스트_사진]
밝게_텐서 = 텐서_사진 * 1.2
```

```
숫자 개수: 6,220,800개
리스트: 0.069초, 텐서: 0.0032초
텐서가 약 21배 빠름
```

리스트는 하나씩, 텐서는 **한꺼번에** 계산한다

NumPy 배열과 거의 같은 그릇에 능력 둘:
GPU에서 계산(4.8절), **계산 이력 기억**(5장 자동 미분)

1차시 · 4장 들어가며 · 예제 04-tensors/b01_why.py (핵심 부분, 시간 재는 줄은 뺐다)

4.1 텐서 만들기: 목록에서, 모양만 정해서

```
기온 = torch.tensor([12.5, 14.1, 13.0])
인원 = torch.tensor([3, 5, 2])
합격 = torch.tensor([True, False, True])
```

```
tensor([12.5000, 14.1000, 13.0000]) torch.float32
tensor([3, 5, 2]) torch.int64
tensor([ True, False,  True]) torch.bool
```

이렇게 부르면	결과	기억할 점
torch.zeros(2, 3)	0으로 채운 2행 3열	빈 그릇 먼저
torch.full((2, 3), 7.0)	7로 채운 2행 3열	원하는 값으로
torch.arange(0, 10, 2)	tensor([0, 2, 4, 6, 8])	간격을 정한다, 끝 빠짐
torch.linspace(0, 1, 5)	tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])	개수를 정한다, 끝 포함

4.1 난수의 씨앗, NumPy와 주고받기

씨앗이 같아도 결과가 다를 때

```
torch.manual_seed(42)
첫번째 = torch.rand(3)

torch.manual_seed(42)
torch.randn(5)
두번째 = torch.rand(3)
```

```
tensor([0.8823, 0.9150, 0.3829])
tensor([0.8694, 0.5677, 0.7411])
```

누가 먼저 난수를 꺼내 가면 순서가 밀린다 → **torch.Generator()**: 나만 쓰는 번호표 기계

복사하는 쪽, 함께 쓰는 쪽

```
배열 = np.array([1.0, 2.0, 3.0])
복사본 = torch.tensor(배열)
공유본 = torch.from_numpy(배열)
배열[0] = 100.0
```

```
NumPy  : [100.   2.   3.]
복사본 : tensor([1., 2., 3.], dtype=torch.float64)
공유본 : tensor([100.,   2.,   3.], dtype=torch.float64)
```

tensor는 옷을 한 벌 더 사기, **from_numpy**는 같은 옷에 이름표 하나 더.
NumPy 기본은 float64 → **.float()**

4.2 모양과 차원: 점, 선, 면, 그리고 책

이름	체온 기록	ndim	shape	numel()
점	체온 하나 (스칼라)	0	()	1
선	하루 세 번 잰 체온 (벡터)	1	(3,)	3
면	이틀치 (행렬)	2	(2, 3)	6
책	일주일치 표 7장	3	(7, 2, 3)	42

```
사진_묶음 = torch.zeros(32, 3, 224, 224)

torch.Size([32, 3, 224, 224])
숫자 4,816,896개, 차원 4개
첫 사진의 모양: torch.Size([3, 224, 224])
```

소리 내어 읽기

32장 · 3색 · 224 × 224

첫 자리는 **배치**: 한꺼번에 처리하는 장 수

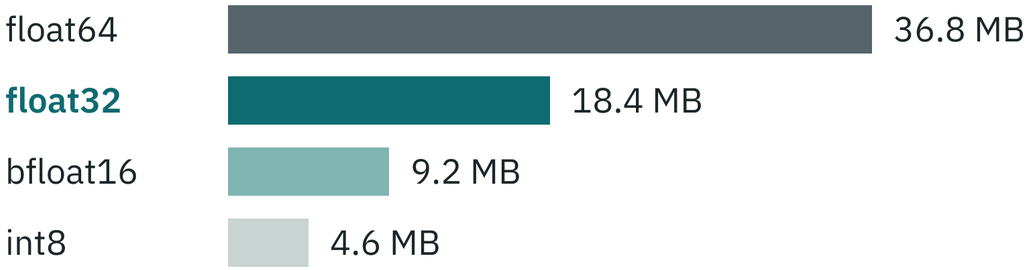
4.3 자료형: 왜 기본이 32비트일까

섞으면 넓은 쪽으로, 정수로 바꾸면 버림

계산	결과
정수 [1, 2, 3] + 실수 [0.5, ...]	torch.float32
tensor([7, 8]) / 2	tensor([3.5000, 4.0000])
tensor([7, 8]) // 2	tensor([3, 4])
tensor([1.5, 2.7]).int()	tensor([1, 2], dtype=torch.int32)

0.1 + 0.2의 실제 값
float32: **0.30000001**192092896 · float64: 0.30000000000000004

사진 32장 묶음의 메모리



메모리 = 칸 크기 element_size() × 칸 개수 numel()

7B 모델: 70억 × 4바이트 = 26.1 GB

4.4 인덱싱: 리스트 문법에 쉼표만 더한다

```
# 3개 도시 × 4분기 평균 기온 (예제용 숫자)
기온 = torch.tensor([[ 2.0, 13.5, 25.7, 15.0],
                     [ 4.5, 15.0, 25.0, 17.5],
                     [ 1.0, 14.0, 26.0, 15.5]])
```

식	결과	뜻
기온[0, 2]	tensor(25.7000)	0번 줄의 2번 칸
기온[:, 2]	tensor([25.7000, 25.0000, 26.0000])	모든 줄의 2번 칸
기온[-1]	tensor([1.0000, 14.0000, 26.0000, 15.5000])	마지막 도시
기온[기온 > 20]	tensor([25.7000, 25.0000, 26.0000])	마스크: 참인 칸만 1차원으로
기온[[0, 2]]	0번, 2번 도시 두 줄	목록으로 고르기

쉼표 앞은 행(도시), 뒤는 열(분기) · **얼음[얼음 < 5] = 0.0**처럼 마스크 자리에 대입하면 조건에 맞는 칸만 한꺼번에 바뀐다

4.4 슬라이스는 복사가 아니라 창이다

```
원본 = torch.arange(6)
조각 = 원본[2:5]
조각[0] = 100
print("조각:", 조각)
print("원본:", 원본)

안전 = 원본[2:5].clone()
안전[0] = -1
print("원본:", 원본)
```

```
조각: tensor([100,   3,   4])
원본: tensor([  0,   1, 100,   3,   4,   5])
원본: tensor([  0,   1, 100,   3,   4,   5])
```

메모리 한 줄과 창



청록 테두리 세 칸이 **조각**: 원본을 들여다보는 창이라, 창으로 칠하면 원본이 칠해진다

- :나 숫자로 자른 것 → 창(공유)
- 마스크·목록으로 고른 것 → 복사
- 따로 쓰고 싶으면 .clone()

4.4 조건으로 바꾸기: where와 clamp

```
print(torch.where(기온 > 20, 1, 0))
print(torch.where(기온 < 5, 0.0, 기온))

print(기온.clamp(min=5.0, max=20.0))
print(torch.tensor([-0.3, 0.5, 1.7]).clamp(0.0, 1.0))
print(torch.tensor([-2.0, 0.0, 3.0]).clamp(min=0.0))
```

- **where(조건, 참일 때, 거짓일 때)**: 원본은 두고 새 표
- **clamp**: 값을 울타리 안에 가둔다
- clamp(min=0.0) = 음수만 0으로 = 6장의 **ReLU**

```
tensor([[0, 0, 1, 0],
        [0, 0, 1, 0],
        [0, 0, 1, 0]])
tensor([[ 0.0000, 13.5000, 25.7000, 15.0000],
        [ 0.0000, 15.0000, 25.0000, 17.5000],
        [ 0.0000, 14.0000, 26.0000, 15.5000]])
tensor([[ 5.0000, 13.5000, 20.0000, 15.0000],
        [ 5.0000, 15.0000, 20.0000, 17.5000],
        [ 5.0000, 14.0000, 20.0000, 15.5000]])
tensor([0.0000, 0.5000, 1.0000])
tensor([0., 0., 3.])
```

1차시 · 4.4 조건으로 바꾸기 · 예제 04-tensors/b26_where.py (핵심 부분, 기온은 앞 장의 표)

4.4 번호표로 꺼내기: 정답 칸의 확률

```
# 사진 4장에 대해 모델이 낸 클래스별 확률 (0: 고양이, 1: 개, 2: 새)
확률 = torch.tensor([[0.7, 0.2, 0.1],
                     [0.1, 0.8, 0.1],
                     [0.3, 0.3, 0.4],
                     [0.6, 0.3, 0.1]])
정답 = torch.tensor([0, 1, 2, 1])            # 장마다 정답 클래스 번호
예측 = 확률.argmax(dim=1)
번호표 = 정답.unsqueeze(1)                   # (4,) → (4, 1): 확률과 차원 수를 맞춘다
정답_확률 = 확률.gather(dim=1, index=번호표)
print(정답_확률.log().neg().mean())
```

```
예측: tensor([0, 1, 2, 0]) 정확도: tensor(0.7500)
tensor([0.7000, 0.8000, 0.4000, 0.3000])
tensor(0.6750)
```

- 줄마다 꺼낼 칸이 달라 슬라이스로는 안 된다
- -log 평균 = 6장의 **교차 엔트로피**

1차시 · 4.4 번호표로 꺼내기: gather · 예제 04-tensors/b27_gather.py (핵심 부분), b28_scatter.py

직접 실행 10분

1. b25_memsize.py의 첫 묶음을 32장에서 64장으로 바꾸면 float32로 몇 MB일까? 먼저 계산하고 실행해 맞춰 본다
2. b11_view_share.py에서 조각 = 원본[::2]로 바꾸면 원본은 어떻게 될까? 안전 = 원본[원본 > 2]로 바꾸면?
3. b27_gather.py에서 정답의 마지막 1을 0으로 바꾸면 정확도와 벌점(-log 평균)은 얼마가 될까?

```
cd examples/vol2/04-tensors && uv run python b11_view_share.py
```

a = torch.arange(6)일 때, a와 메모리를 함께 쓰는 것은?

Q

가 a[2:5]

나 a[a > 2]

다 a[[0, 2]]

라 a[2:5].clone()

1차시 정리

텐서는 NumPy 배열 같은 그릇에 GPU 계산과 계산 이력 기억을 더한 것. 한꺼번에 계산해서 빠르다

모양은 자리마다 이름 붙여 읽는다: (32, 3, 224, 224) = **32장 · 3색 · 224 × 224**. 기본 실수는 **float32**

슬라이스는 **창**(공유), 마스크·목록은 **복사**. 따로 쓰려면 **.clone()**

다음 차시: 모양을 바꾸고, 반복문 없이 한꺼번에 계산하기

2차시

02

모양 바꾸기와 한꺼번에 계산하기

책 4.5 ~ 4.11절

학습 목표

- reshape·permute·unsqueeze로 모양을 바꾼다
- 브로드캐스팅·행렬 곱·dim 집계로 반복문 없이 계산한다
- 장치를 옮기고 저장하며, 오류 메시지를 읽고 고친다

4.5 숫자는 그대로, 모양만 바꾸기

코드	결과	하는 일
torch.arange(12).reshape(2, -1)	2행 6열	접고 펴기, -1은 알아서
사진.permute(2, 0, 1)	(224, 224, 3) → (3, 224, 224)	차원 순서 바꾸기
사진.unsqueeze(0)	torch.Size([1, 3, 224, 224])	1장짜리 묶음으로 포장
묶음.squeeze(0)	torch.Size([3, 224, 224])	포장 풀기
torch.cat([a, b])	tensor([1, 2, 3, 4, 5, 6])	이어 붙이기, 차원 수 그대로
torch.stack([a, b])	torch.Size([2, 3])	새 차원을 만들어 쌓기
zeros(32, 3, 4, 4).flatten(start_dim=1)	torch.Size([32, 48])	사진마다 한 줄로 (9장)

```
x.reshape(5, -1)  →  오류: shape '[5, -1]' is invalid for input of size 12
```

4.5 메모리는 한 줄, 보폭(stride)으로 표처럼 읽는다



행렬 · 보폭 (3, 1)

```
0 1 2
3 4 5
```

아랫줄 +3, 오른쪽 +1

전치 .T · 보폭 (1, 3)

```
0 3
1 4
2 5
```

아랫줄 +1, 오른쪽 +3

```
같은 메모리? True
이어져 있나? False
view(6) → 오류
reshape(6) → [0, 3, 1, 4, 2, 5]
```

전치는 숫자를 옮기지 않고 보폭만 바꾼다. contiguous()는 보이는 순서대로 새로 복사

4.6 브로드캐스팅: 작은 쪽을 펼쳐서 맞춘다

점수 (3, 2): 학생 3명 × 과목 2개 = [[80, 70], [92, 88], [75, 95]]

+ 과목별 [3, 10] · (2,)

```
[[ 83.,  80.],
 [ 95.,  98.],
 [ 78., 105.]]
```

줄마다 똑같이 → (3, 2)

+ 학생별 [[1], [2], [3]] · (3, 1)

```
[[81., 71.],
 [94., 90.],
 [78., 98.]]
```

칸마다 똑같이 → (3, 2)

+ [1, 2, 3] · (3,)

오류: The size of tensor a (2) must match the size of tensor b (3) at non-singleton dimension 1

① 두 모양을 **오른쪽 끝부터** 맞댄다 ② 길이가 **같거나 한쪽이 1**이면 통과 ③ 자리가 없으면 1로 본다

4.6 행렬 곱: 곱하고 더하기를 한꺼번에

```
# 학생 3명 × 과목 2개 점수, 과목별 반영 비율
점수 = torch.tensor([[80.0, 70.0],
                     [92.0, 88.0],
                     [75.0, 95.0]])
비율 = torch.tensor([0.4, 0.6])

print(점수 @ 비율)                      # (3, 2) @ (2,) → (3,)
print((점수 * 비율).sum(dim=1))         # 같은 계산을 풀어 쓰면
```

```
tensor([74.0000, 89.6000, 87.0000])
tensor([74.0000, 89.6000, 87.0000])
```

(a, b) @ (b, c) → (a, c)

가운데 b가 같아야 하고, 곱하고 더하면서 사라진다. 신경망의 층 = **입력 @ 가중치 + 편향**

4.6 집계: dim은 그 차원이 사라진다

점수 (3, 2)에서	결과	뜻
점수.mean(dim=0)	tensor([82.3333, 84.3333])	과목별 평균
점수.mean(dim=1)	tensor([75., 90., 85.])	학생별 평균
점수.mean(dim=1, keepdim=True).shape	torch.Size([3, 1])	1로 남겨 두기
점수.max(dim=0)	값 [92., 95.], 위치 [1, 2]	과목별 최고와 그 학생
점수.argmax(dim=1)	tensor([0, 0, 1])	학생별 잘한 과목

keepdim=True면 (3, 1)이라 학생 줄마다 바로 브로드캐스팅된다. (3,)이면 오른쪽 끝에 맞춰져 오류

topk(3): 점수 [72, 95, 88, 60, 91, 79]에서 값 [95., 91., 88.], 번호 [1, 4, 2]. 상위 5개 정확도가 이 계산

4.7 제자리 연산, 그리고 공유와 복사 한눈에

```
a = torch.tensor([1.0, 2.0, 3.0])
b = a.add(10)
print(a, b)
a.add_(10)
print(a)
```

```
tensor([1., 2., 3.])
tensor([11., 12., 13.])
tensor([11., 12., 13.])
```

밑줄 _ = 원본 자체를 바꾼다. 5장 자동 미분과 섞으면 오류를 만나기 쉬우니 처음엔 밑줄 없는 쪽

이렇게 만들면	원본과의 관계
from_numpy(배열), 텐서.numpy() (CPU)	공유
텐서[1:3], 텐서[:, 0] 같은 슬라이스	공유
view, permute, .T, unsqueeze, squeeze	공유
reshape	되면 공유, 안 되면 복사
torch.tensor(...), .clone()	복사
마스크·목록으로 고른 결과, cat, stack	복사
.contiguous()	이어져 있으면 그대로, 아니면 복사
텐서.add_(...) 같은 밑줄 함수	원본 자체를 바꿈

4.8 · 4.9 장치로 옮기고, 파일로 저장하기

cuda → mps → cpu 순서로 고르기

```
장치 = 장치_고르기()
print("고른 장치:", 장치)
x = torch.ones(3)
y = x.to(장치)
print(y.device)
```

```
고른 장치: mps
mps:0
```

.to()는 **새 텐서**를 돌려준다. 4096 × 4096 행렬 곱: CPU 0.057초, MPS 0.011초

torch.save와 torch.load

```
torch.save(기온, 경로)
다시 = torch.load(경로)
torch.save(큰_텐서[:10], 조각_경로)
torch.save(큰_텐서[:10].clone(), 복사_경로)
torch.load(경로, map_location="cpu")
```

```
조각.pt 4,001,577바이트
복사.pt 1,577바이트
```

슬라이스는 창 너머 메모리 전체가 저장된다 → **clone()** 후 저장

오류: Expected all tensors to be on the same device, but found at least two devices, mps:0 and cpu!

2차시 · 4.8 장치, 4.9 저장하고 불러오기 · 예제 04-tensors/b20~b22, b32_save.py, b33_map_location.py (핵심 부분, 경로 이름은 줄였다)

4.10 오류 메시지는 숫자와 자료형부터 읽는다

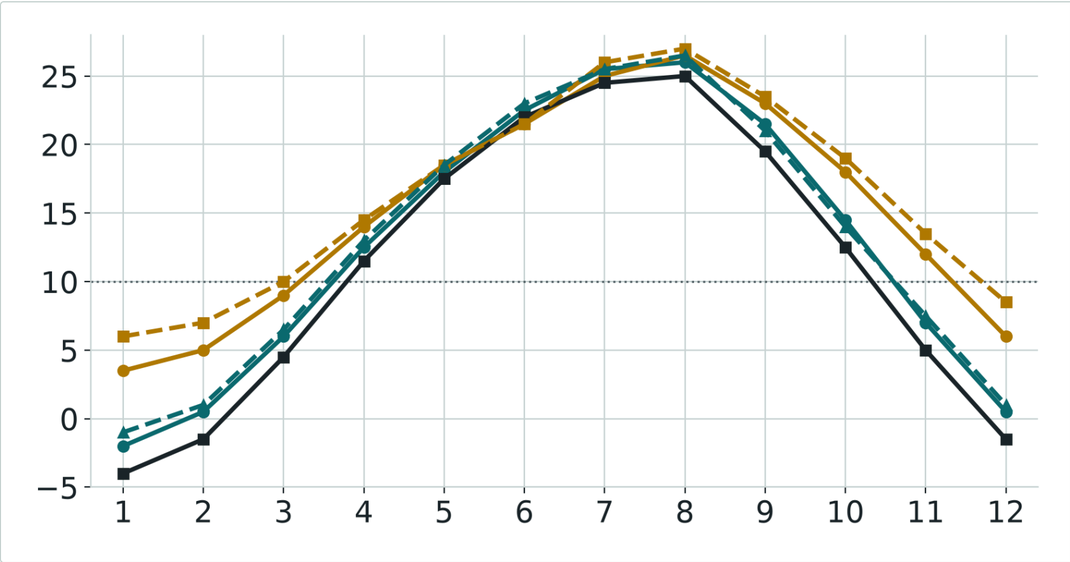
오류 첫 줄	뜻	고치기
mat1 and mat2 shapes cannot be multiplied (4x3 and 4x2)	가운데 3과 4가 다르다	w를 (3, 2)로
expected m1 and m2 to have the same dtype, but got: long long != float	정수와 실수를 행렬 곱	.float()
a Tensor with 3 elements cannot be converted to Scalar	.item()은 숫자 하나만	sum() 또는 .tolist()
stack expects each tensor to be equal size, but got [3] at entry 0 and [4] at entry 1	쌓을 모양이 다르다	길이 맞추기 (7장)

고친 뒤엔 shape를 찍어 본다:
torch.Size([4, 3]) @ torch.Size([3, 2]) → torch.Size([4, 2])

조용한 함정, 표준편차 [2, 4, 4, 6]:
np.std 1.4142 · torch std 1.6330 (n − 1로 나눔)

4.11 실습: 도시별 기후 패턴 비교하기

```
표준화 = (기온 - 평균) / 표준편차  # (5, 12) - (5, 1) → 브로드캐스팅
유사도 = 단위 @ 단위.T  # (5, 12) @ (12, 5) → (5, 5)
```



가로 1~12월 · ● 가람 ▲ 마루(점선) · ■ 다솜 · ● 나래 ■ 라온(점선) · 점선 10°C

도시	연평균	연교차	닮은	난방
가람시	12.71	28.0	마루	38.0
나래시	15.17	23.0	라온	16.5
다솜시	11.25	29.0	마루	47.5
라온시	16.25	21.0	나래	8.5
마루시	13.04	27.5	다솜	35.0

직접 실행 10분

1. b23_practice.py의 난방 기준을 10도에서 5도로 바꾼다(세 군데). 난방이 필요한 달 수와 난방 지수는?
2. 가장 추운 달(argmin)과 냉방 지수(22도보다 더운 만큼, clamp)를 한 줄씩 더해 출력한다
3. 평균 = 기온.mean(dim=1, keepdim=True)에서 keepdim=True를 빼면 무슨 일이 생길까? 먼저 예상하고 실행한다

```
cd examples/vol2/04-tensors && uv run python b23_practice.py
```

2차시 · 직접 실행 · 원본은 고치지 말고 복사본(예: 내_기후.py)에서 · 1초 안쪽으로 끝난다

모양이 (3, 2)인 텐서에 더할 때 오류가 나는 것은?

Q

가 모양 (2,)인 텐서

나 모양 (3, 1)인 텐서

다 모양 (3,)인 텐서

라 숫자 5

2차시 정리

reshape는 줄을 바꿔 늘어놓고 **permute**는 차원 순서만 바꾼다. 메모리는 한 줄, 보폭으로 표처럼 읽는다

브로드캐스팅은 **오른쪽부터, 같거나 1이면 통과**. (a, b) @ (b, c) → (a, c). **dim**은 그 차원이 사라진다

같은 장치끼리만 계산, 슬라이스는 clone() 후 저장, 오류는 숫자와 자료형부터 읽고 shape를 찍는다

다음 차시: 5장, 안개 낀 산에서 내려오는 법 · 기울기와 경사하강법

3차시

03

기울기와 손으로 하는 경사하강법

책 5장 들어가며 ~ 5.3절

학습 목표

- 손실(평균제곱오차)이 무엇을 재는지 말한다
- 기울기를 수치 미분으로 재고 부호와 크기를 읽는다
- 경사하강법으로 함수의 바닥과 직선의 답을 손으로 찾는다

5장 안개 낀 산에서 내려오려면

판매량 = w × 기온 + b, 아무 직선이나 그어 보면

```
기온   : [28.2 22.3 18.7 18.3 31. ]
판매량: [51. 40. 31. 33. 63.]
w=2.0, b=   0.0 → 오차   20.76
w=2.0, b=  -5.0 → 오차   39.62
w=2.5, b= -10.0 → 오차   29.22
w=3.0, b= -20.0 → 오차   72.05
```

손실 = 평균((예측 − 실제)^2), 평균제곱오차. 찍어서는 끝이 없다

산 비유	학습
땅 위의 위치	(w, b)
그 자리의 높이	손실
골짜기 바닥	가장 좋은 직선
발끝 감각	기울기(미분)
내리막으로 한 걸음	경사하강법

5.1 기울기: 조금 움직였을 때 얼마나 변하나

h	잰 기울기
1.0	7.000000
0.1	6.100000
0.01	6.010000
0.0001	6.000100
1e-08	6.000000
1e-12	6.000533
1e-14	6.039613

```
x = 3.0      # f(x) = x ** 2
기울기 = (f(x + h) - f(x)) / h
```

h를 줄이면 6에 다가간다. 그런데 너무 작으면 실수 오차가 부풀어 오히려 틀어진다

양쪽으로 움직이기 (중앙 차분)

```
(f(x + h) - f(x - h)) / (2 * h)
```

h = 0.01에서 한쪽만 6.010000, 양쪽 6.000000

3차시 · 5.1 기울기 · 예제 05-autograd/b02_numeric_slope.py (출력을 표로, 일부 줄), b03_central.py

5.1 여러 방향의 기울기, 그래디언트

이 장에서 쓸 공식은 몇 개뿐

함수	기울기	말로 풀면
c (상수)	0	변하지 않는다
a × x	a	a배만큼 변한다
x^2	2x	멀리 갈수록 가파름
f(x) + g(x)	기울기의 합	따로 구해 더한다

부호: **양수**는 오른쪽으로 오르막, **음수**는 내리막, 0은 평평한 바닥

높이 = $x^2 + 3y^2$, (1, 2)에서

```
동서 방향 기울기: 2.0000  (공식 2x = 2.0)
남북 방향 기울기: 12.0000  (공식 6y = 12.0)
지금 높이: 13.0000
기울기 반대로 한 걸음: 11.5636
기울기 방향으로 한 걸음: 14.5236
```

그래디언트 (2, 12)는 가장 가파른 **오르막**. 내려가려면 **반대쪽**으로

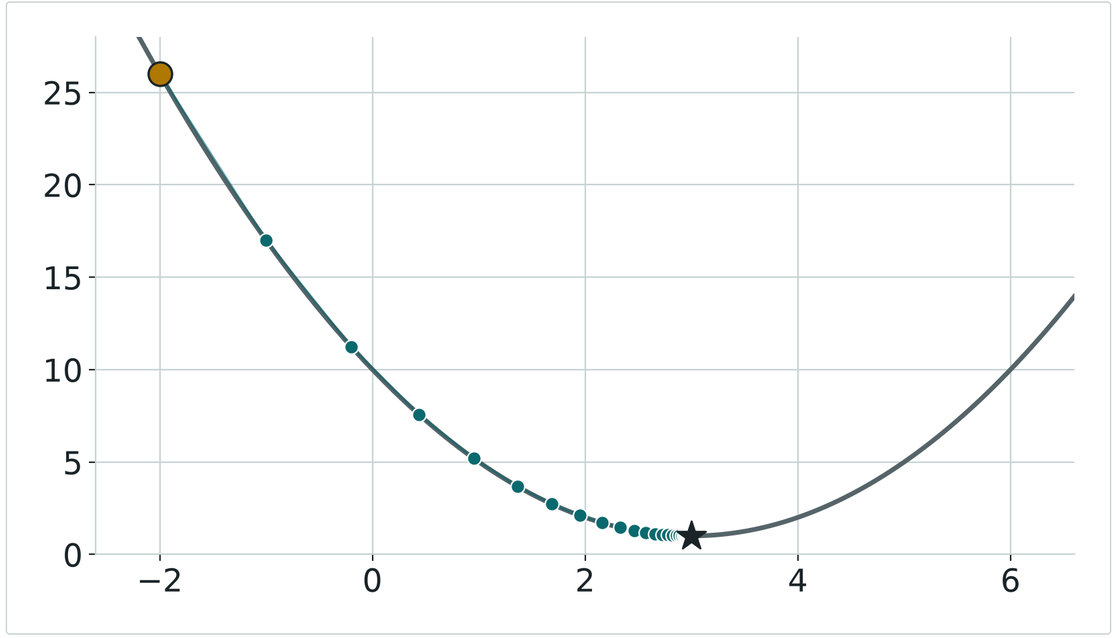
5.2 한 걸음씩 바닥으로

새 위치 = 지금 위치 − 학습률 × 지금 자리의 기울기

```
def f(x: float) -> float:
    return (x - 3) ** 2 + 1

x = -2.0
학습률 = 0.1
for 걸음 in range(1, 31):
    x = x - 학습률 * 기울기(x)
```

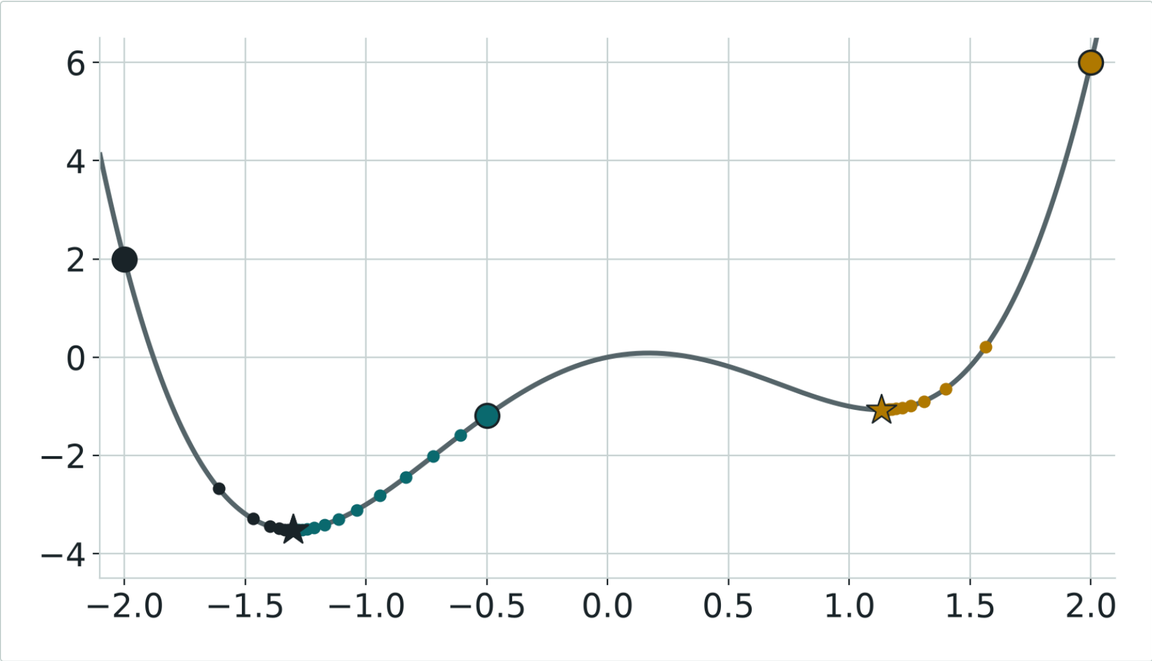
```
 1걸음: x = -1.0000, f(x) = 17.000000
 2걸음: x = -0.2000, f(x) = 11.240000
10걸음: x = 2.4631, f(x) = 1.288230
30걸음: x = 2.9938, f(x) = 1.000038
```



가로 x · 세로 f(x) · ● 출발 −2 · ● 한 걸음씩 · ★ 바닥 (3, 1)

3차시 · 5.2 손으로 해 보는 경사하강법 · 예제 05-autograd/b05_gd_1d.py (핵심 부분, 출력 일부) · 그림은 같은 설정으로 그렸다

5.2 공식이 없어도 내려간다, 하지만 어느 골짜기로?



큰 점 = 출발, ★ = 200걸음 뒤 · 2.0 · −0.5 · −2.0

```
x = x - 0.01 * 수치_미분(f, x)
```

```
출발  2.0: 도착 x =  1.1309
출발 -0.5: 도착 x = -1.3008
출발 -2.0: 도착 x = -1.3008
```

지역 최솟값: 주변보다만 낮은 곳
전역 최솟값: 진짜 가장 낮은 곳

선형 회귀의 손실은 골짜기가 **하나**인 그릇이라 걱정 없다

3차시 · 5.2 공식이 없어도 내려갈 수 있습니다 · 예제 05-autograd/b06_local_min.py (핵심 부분, 출력에서 f(x)는 뺐다) · 그림은 같은 설정으로 그렸다

5.3 손실을 w와 b로 미분하고, 재서 맞춰 보기

손실

평균($(w \times x + b - y)^2$)

$w \times x + b - y$ = 날마다의 **오차**

w 방향 기울기

평균($2 \times$ 오차 $\times x$)

기온 높은 날의 오차가 크게 반영

b 방향 기울기

평균($2 \times$ 오차)

많이 예측했으면 b를 줄인다

```
w 방향: 수치 미분 -25.3119, 공식 -25.3119
b 방향: 수치 미분 -105.0000, 공식 -105.0000
```

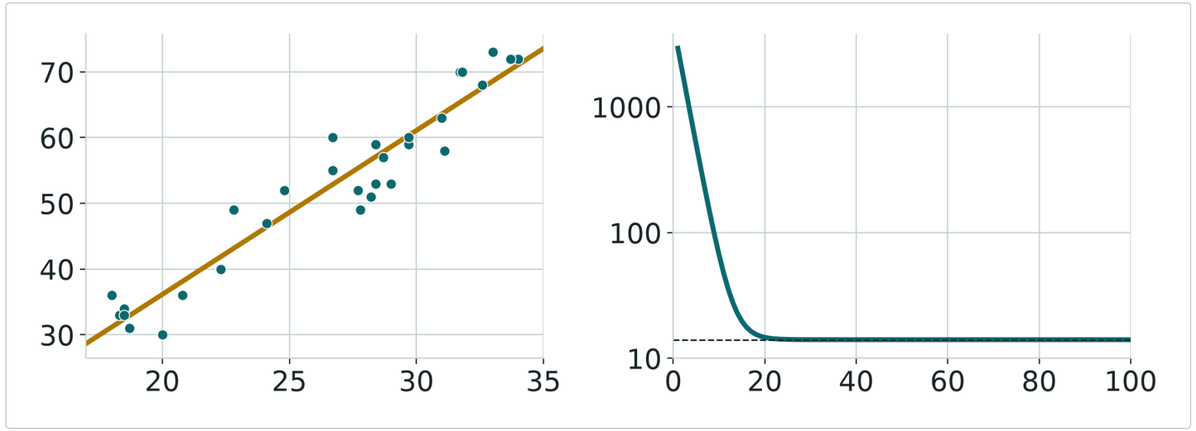
기울기 검사: 공식과 잰 값이 같다. 둘 다 음수 → w와 b를 **키우는** 쪽으로

기온(18~34)은 2장처럼 **표준화**해서 쓴다: $x = (\text{기온} - \text{평균}) / \text{표준편차}$. 이유는 5차시 5.10절에서 숫자로

5.3 경사하강법으로 직선 찾기

```
w, b = 0.0, 0.0
학습률 = 0.1
for 반복 in range(1, 101):
    오차 = w * x + b - y
    손실 = (오차 ** 2).mean()
    기울기_w = (2 * 오차 * x).mean()
    기울기_b = (2 * 오차).mean()
    w -= 학습률 * 기울기_w
    b -= 학습률 * 기울기_b
```

```
  1번째: 손실 2930.500, w = 2.5312, b = 10.5000
 10번째: 손실   66.615, w = 11.2970, b = 46.8629
 20번째: 손실   14.683, w = 12.5100, b = 51.8947
100번째: 손실   14.077, w = 12.6559, b = 52.5000
polyfit 정답: w = 12.6559, b = 52.5000
기온 1°C당 판매량 변화: 2.495개
```



왼쪽: 기온(°C)과 판매량, — 찾은 직선 · 오른쪽: 반복별 손실, 점선 14.077

100걸음 걸어 찾은 답 = 공식(**polyfit**, 최소제곱법)의 답. 신경망엔 그런 공식이 없어서 경사하강법을 쓴다

3차시 · 5.3 경사하강법으로 직선 찾기 · 예제 05-autograd/b08_numpy_gd.py (핵심 부분, 출력 일부) · 그림은 같은 설정으로 그렸다

직접 실행 10분

1. b05_gd_1d.py의 학습률을 0.5로, 다음엔 1.0으로 바꿔 실행한다. 30걸음 동안 x는 어떻게 움직이나?
2. b06_local_min.py의 출발점을 [0.0, 0.2, 0.5]로 바꾸면 각각 어느 골짜기에 도착할까? 그림을 보고 먼저 예상한다
3. b08_numpy_gd.py의 학습률을 0.01로 줄이면 100번째 손실은? 바닥 14.077에 닿았나?

```
cd examples/vol2/05-autograd && uv run python b05_gd_1d.py
```

x = 3에서 기울기가 6이다. 학습률 0.1로 한 걸음 가면 x는?

Q

가 3.6

나 2.4

다 2.9

라 -3

3차시 정리

손실은 모델이 얼마나 틀렸는지를 숫자 하나로 잰 것. 학습 = 손실 산의 **바닥 찾기**

기울기 = 조금 움직였을 때 변한 양 ÷ 움직인 거리. 그래디언트의 **반대쪽**이 가장 빠른 내리막

새 위치 = 지금 위치 − 학습률 × 기울기. 선형 회귀에서는 공식(polyfit)과 같은 답에 도착

다음 차시: 기울기 공식을 PyTorch에게 맡긴다 · 자동 미분

4차시

04

PyTorch에게 미분 맡기기

책 5.4 ~ 5.7절

학습 목표

- requires_grad · backward · grad로 손으로 구한 기울기를 다시 얻는다
- 계산 그래프와 연쇄 법칙으로 역전파를 설명한다
- .grad가 쌓이는 함정을 알고 학습 반복의 네 단계를 지킨다

5.4 requires_grad, backward, grad

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
print(y)

y.backward()
print("PyTorch가 구한 기울기:", x.grad)
```

```
tensor(9., grad_fn=<PowBackward0>)
PyTorch가 구한 기울기: tensor(6.)
손으로 구한 기울기 2x: 6.0
수치 미분: 5.999999999999339
```

1 표시하기

requires_grad=True: 거쳐 갈 계산을 기억해 줘

2 계산하기

grad_fn=<PowBackward0>가 그 기억

3 거꾸로 계산하기

backward() → 기울기가 x.grad에

5.4 손으로 구한 기울기와 똑같을까

```
# PyTorch에게 맡긴 기울기
x = torch.from_numpy(x_np)                    # float64 그대로 (4.1절)
y = torch.from_numpy(y_np)
w = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
b = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
손실 = ((w * x + b - y) ** 2).mean()
손실.backward()
```

```
손으로   : w 기울기 -25.3118789700, b 기울기 -105.0000000000
PyTorch : w 기울기 -25.3118789700, b 기울기 -105.0000000000
같은가? True True
```

손실 한 줄과 backward() 한 줄뿐, 기울기 공식은 어디에도 없다. 그런데 3차시에 공들여 유도한 공식과 **소수점 열째 자리까지** 같다

4차시 · 5.4 손으로 구한 기울기와 똑같을까요? · 예제 05-autograd/b10_same_grad.py (핵심 부분)

5.4 처음에 꼭 만나는 규칙 세 가지

backward()는 숫자 하나에서	오류 1: grad can be implicitly created only for scalar outputs y.sum().backward() → x.grad: tensor([2., 4., 6.])
실수 텐서만 기울기를	torch.tensor([1, 2, 3], requires_grad=True) 오류 2: Only Tensors of floating point and complex dtype can require gradients
기울기는 잎에만 남는다	x는 잎인가? True / y는 잎인가? False y.grad: None

경사하강법은 언제나 **손실이라는 숫자 하나**를 줄이는 일. 고칠 대상(w, b)은 잎이라 그걸로 충분하다

5.5 계산 그래프: PyTorch는 무엇을 기억하나

```
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
x = torch.tensor(3.0)
y = torch.tensor(10.0)

예측 = w * x + b
손실 = (예측 - y) ** 2
그래프_따라가기(손실.grad_fn)
```

grad_fn.next_functions를 재귀로 끝까지 따라간다

```
손실: tensor(9., grad_fn=<PowBackward0>)
PowBackward0
    SubBackward0
        AddBackward0
            MulBackward0
                AccumulateGrad  ← 잎 텐서 값 2.0
            AccumulateGrad  ← 잎 텐서 값 1.0
```

아래에서 위로 읽으면 쓴 순서: 곱하기 → 더하기 → 빼기 → 제곱. 데이터 x, y는 기록하지 않는다

4차시 · 5.5 grad_fn을 따라가 보기 · 예제 05-autograd/b12_graph.py (핵심 부분, 그래프_따라가기 함수 본문은 뺐다)

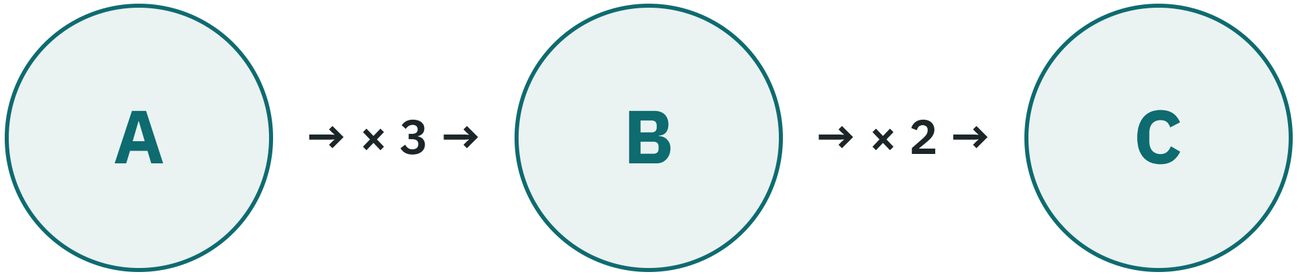
5.5 그래프는 계산할 때마다 새로 만들어진다

```
def 요금(사용량: torch.Tensor) -> torch.Tensor:
    """100 넘게 쓰면 넘은 만큼은 단가가 두 배가 되는 누진 요금."""
    if 사용량 > 100:                              # 평범한 파이썬 if
        return 100 * 1.0 + (사용량 - 100) * 2.0
    return 사용량 * 1.0
```

```
사용량 80.0: 요금 기울기 1.0
사용량 130.0: 요금 기울기 2.0
오류: Trying to backward through the graph a second time
```

- 지나간 길만 기록: **실행하면서 정의**
- 그래프는 **일회용**: 반복마다 손실을 새로 계산

5.6 연쇄 법칙: 톱니바퀴 세 개



A가 한 바퀴 돌면 C는 3 × 2 = **6바퀴**. 끝까지의 기울기 = **단계마다 기울기의 곱**

```
x값 = 2.0
u값 = 3 * x값 + 1
du_dx = 3.0
dy_du = 2 * u값
```

```
연쇄 법칙(손): dy/dx = 14.0 × 3.0 = 42.0
PyTorch       : dy/dx = 42.0
수치 미분     : dy/dx = 42.0000
```

5.6 역전파를 손으로 따라 하기

앞으로: x = 3, y = 10, w = 2, b = 1 → 예측 = w × x + b = 7 → 오차 = 예측 − y = −3 → 손실 = 오차² = 9

거꾸로, 끝에서부터	지역 기울기	곱해 온 값 (손)	PyTorch
손실 = 오차²	2 × 오차	d손실/d오차 = −6	-6.0
오차 = 예측 − y	1	d손실/d예측 = −6	-6.0
예측 = w × x + b (w)	x = 3	d손실/dw = −18	-18.0
예측 = w × x + b (b)	1	d손실/db = −6	-6.0

왜 거꾸로? 공통 부분(−6)을 한 번만 계산해 모든 갈래에 나눠 준다. 수백만 개도 backward() 한 번

중간 결과의 기울기는 retain_grad()로 남겨 달라고 해서 비교했다

5.7 backward를 부를 때마다 더해진다

```
x = torch.tensor(3.0, requires_grad=True)
for 번째 in range(1, 4):
    y = x ** 2
    y.backward()
    print(f"{번째}번째 backward 뒤 x.grad = {x.grad.item()}")

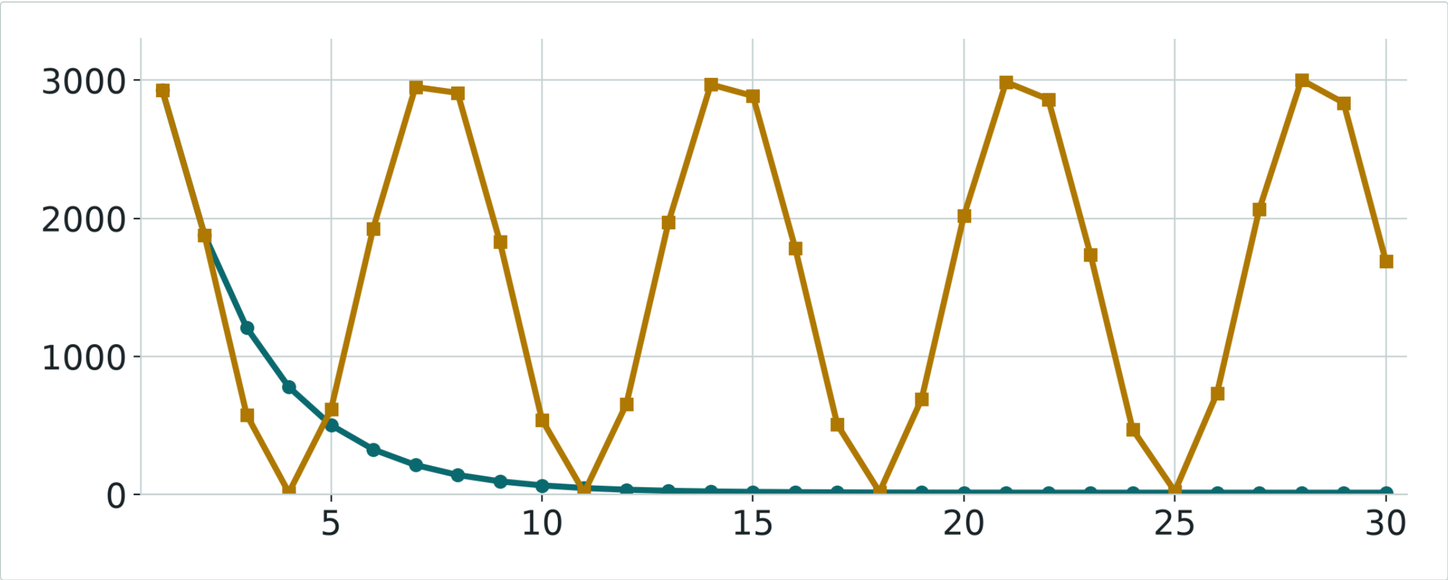
x.grad.zero_()                  # 비우기
(x ** 2).backward()
print("비운 뒤 다시:", x.grad.item())
```

```
1번째 backward 뒤 x.grad = 6.0
2번째 backward 뒤 x.grad = 12.0
3번째 backward 뒤 x.grad = 18.0
비운 뒤 다시: 6.0
```

기울기는 언제나 6인데 6, 12, 18로 불었다. .grad에 **덮어쓰지 않고 더한다**

일부러 그렇게 만들었다
여러 갈래로 쓰인 텐서는 갈래마다 온 기울기를 더해야 맞다(y = x * x). 작은 배치 여럿을 쌓아 큰 배치처럼 쓰는 기법(17장)도 이 덕분

5.7 비우지 않으면 조용히 학습이 망가진다



가로 반복 1~30 · 세로 손실 · ● 매번 비움 → 14.1 · ■ 비우지 않음 → 1691.5

```
grad 비우기=True : [2930.5, 327.2, 47.7, 17.7, 14.5, 14.1] → 14.1
grad 비우기=False: [2930.5, 1927.2, 15.8, 1786.5, 2987.0, 734.2] → 1691.5
```

학습 반복의 순서

1. 손실 계산 (새 그래프)
2. backward()
3. no_grad 안에서 고치기
4. 기울기 비우기

6장에서 3·4번은 optimizer.step()과 zero_grad() 두 줄이 된다

직접 실행 10분

1. b15_backprop.py를 x = 2, y = 5, w = 1, b = 0으로 바꾼다. d손실/dw와 d손실/db를 먼저 손으로 구한 뒤 실행해 맞춰 본다
2. b14_chain_hand.py를 $y = (2x - 1)^3$, x = 1로 바꾼다. PyTorch와 수치 미분 줄만 고치면 손 계산과 어긋난다. 손 계산 줄도 고쳐 맞춘다
3. b16_accumulate.py를 다섯 번 반복하게 바꾸면 마지막 x.grad는?

```
cd examples/vol2/05-autograd && uv run python b15_backprop.py
```

4차시 · 직접 실행 · 세 파일 모두 1초 안팎 · 손 계산을 먼저 종이에 적고 나서 실행한다

학습 반복에서 기울기 비우기를 빠뜨리면?

Q

가 오류가 나서 멈춘다

나 오류 없이 손실이 출렁인다

다 기울기가 0이 되어 멈춘다

라 아무 영향이 없다

4차시 정리

requires_grad=True → 평소처럼 계산 → **backward()** → **.grad**. 손으로 유도한 공식과 같은 값

그래프는 실행하면서 만들어지고, 역전파는 끝에서부터 **지역 기울기를 곱해** 내려간다(연쇄 법칙)

.grad는 쌓인다. 손실 → backward → no_grad에서 고치기 → 비우기

다음 차시: 기록 끄기, 학습률과 미니배치, 당뇨병 데이터로 처음부터 학습

5차시

05

학습률, 미니배치, 그리고 첫 실전 학습

책 5.8 ~ 5.13절

학습 목표

- no_grad · inference_mode · detach를 언제 쓰는지 가른다
- 학습률과 표준화가 학습에 주는 영향을 손실로 읽는다
- 미니배치 경사하강법으로 실제 데이터 회귀를 처음부터 학습한다

5.8 값을 고칠 때는 기록을 끈다

```
w -= 0.01 * w.grad                      # 기록을 켠 채로 잎 텐서를 고치면?
w2 = w - 0.01 * w.grad                  # 새 텐서를 만들면 오류는 없지만...
with torch.no_grad():                   # 올바른 방법
    w -= 0.01 * w.grad
```

```
오류: a leaf Variable that requires grad is being used in an in-place operation.
w2: tensor(2.2400, grad_fn=<SubBackward0>) / 잎인가? False
w: tensor(2.2400, requires_grad=True) / 잎인가? True
```

방법	범위	주로 쓰는 곳
with torch.no_grad():	블록 전체	파라미터 고치기, 검증 데이터로 평가
with torch.inference_mode():	블록 전체	학습이 끝난 모델로 예측만
텐서.detach()	텐서 하나	NumPy로 넘기기, 기록·그래프 그리기

5차시 · 5.8 기록 끄기: no_grad, detach, inference_mode · 예제 05-autograd/b18_update_error.py (핵심 부분), b19_no_grad.py

5.9 제자리 연산과 자동 미분의 충돌

```
a = torch.tensor([0.0, 1.0, 2.0], requires_grad=True)
b = a.exp()      # exp는 기울기 계산에 '자기 결과'를 다시 쓴다
print("b의 버전:", b._version)
b.add_(1)        # 결과를 제자리에서 바꿔 버리면?
print("b의 버전:", b._version)
b.sum().backward()
```

`c = b + 1` 밑줄 없이 새 텐서 → 기울기 [1.0000, 2.7183, 7.3891]

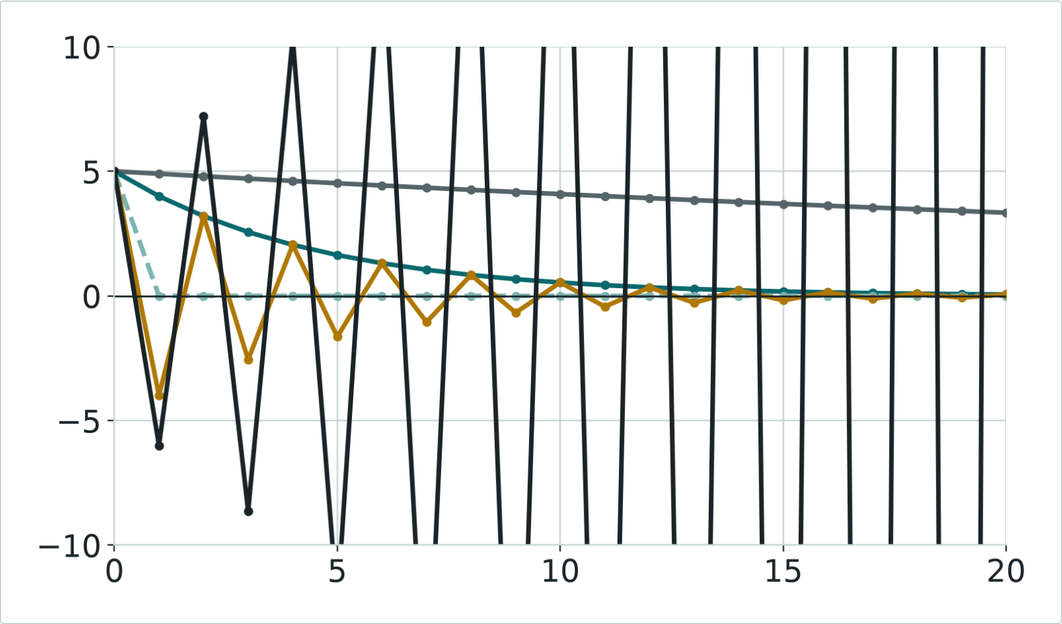
`b = a * 3` 뒤 add_는 괜찮다 → [3., 3., 3.]. 곱하기 3의 기울기는 b 값과 무관

```
b의 버전: 0
b의 버전: 1
오류: one of the variables needed for gradient computation has been modified by an inplace operation:
[torch.FloatTensor [3]], which is output 0 of Exp, is at version 1; expected version 0 instead.
```

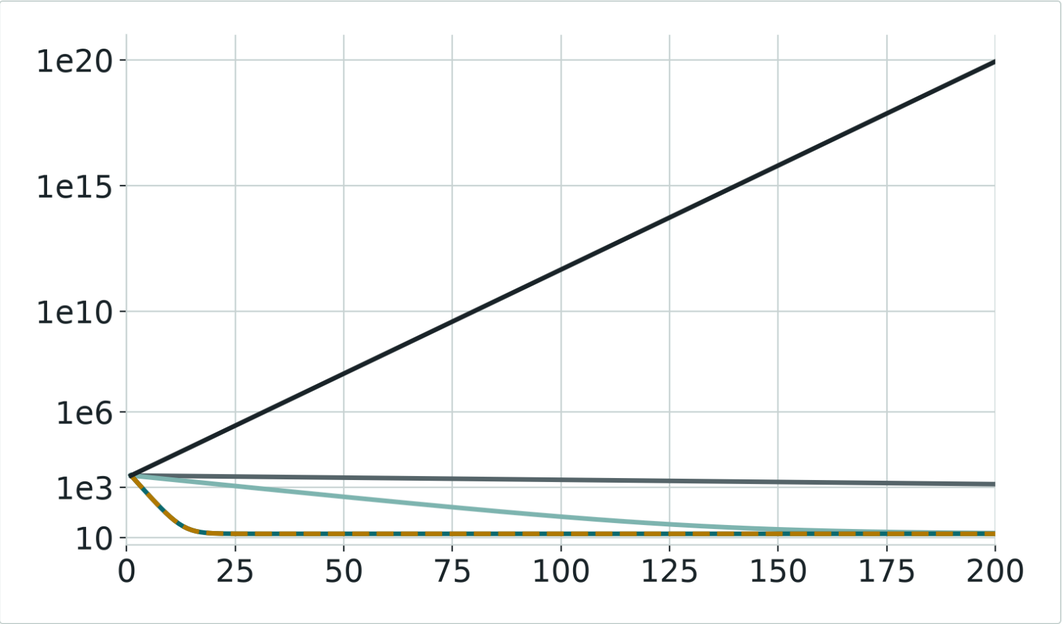
5차시 · 5.9 제자리 연산과 자동 미분의 충돌 · 예제 05-autograd/b20_inplace.py (핵심 부분, 긴 오류 줄은 화면에서 둘로 나눴다)

5.10 학습률: 너무 크면 튀고, 너무 작으면 늦다

f(x) = x², x = 5에서 20걸음 · 세로는 x



아이스크림 회귀 손실 200번 · 로그 눈금



왼쪽 학습률 ■ 0.01 · ■ 0.1 · ■ 0.5(옅은 점선) · ■ 0.9 · ■ 1.1 | 오른쪽 ■ 0.001 · ■ 0.01(옅은) · 0.1 · ■ 0.9(점선) · ■ 1.05

5차시 · 5.10 학습률 · 예제 05-autograd/b21_lr_1d.py, b22_lr_regression.py · 그림은 같은 설정으로 모든 걸음을 그렸다

5.10 표준화하지 않으면 생기는 일

```
x_원래 = torch.tensor(기온, dtype=torch.float32)          # 표준화하지 않은 18~34

학습률 0.01,    5번째: 손실 3.484e+12, w = 940409.688, b = 34166.535
학습률 0.01,  100번째: 손실 nan, w = nan, b = nan
학습률 0.001,    5번째: 손실 27.05, w = 2.035, b = 0.069
학습률 0.001, 5000번째: 손실 17.41, w = 2.141, b = -4.027
정답: w = 2.495, b = -13.758
```

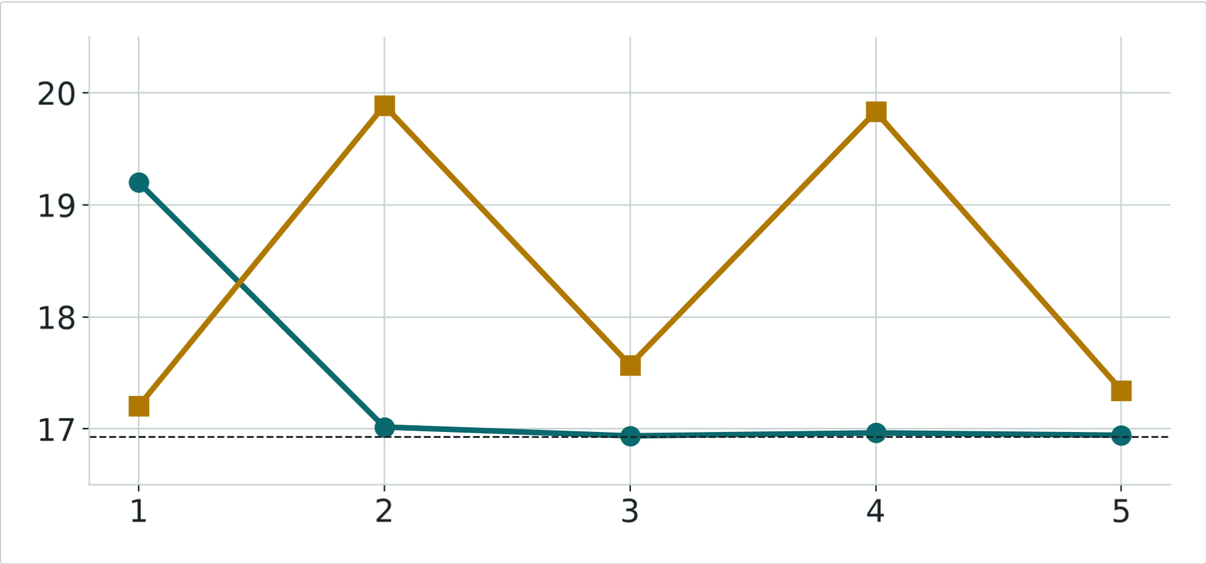
0.01: 표준화했을 땐 무난했는데 100번째에 nan. w 방향 경사가 수백 배 가파르다

0.001: 발산은 멈췄지만 5000번에도 b = −4.027. 완만한 b 방향은 한없이 느리다

표준화는 길쭉한 협곡을 **둥근 그릇**으로 만들어 학습률 하나로 잘 내려가게 한다

5차시 · 5.10 표준화하지 않으면 생기는 일 · 예제 05-autograd/b23_scale.py (핵심 한 줄과 출력 일부)

5.11 미니배치: 전부 볼까, 조금씩 볼까



가로 에폭 · 세로 전체 손실(16.5~20.5 확대) · ● 배치 32 · ■ 배치 1 · 점선 16.93 · 배치 1000은 그림 위 밖

배치 1000 · 5번 고침
5에폭 뒤 944.36

배치 32 · 160번 고침
16.94, 바닥 16.93 바로 위

배치 1 · 5000번 고침
17.20 ~ 19.89로 흔들림

에폭 = 데이터 전체를 한 바퀴. 같은 5에폭인데 결과가 다르다

5.12 torch.autograd.grad: 기울기를 직접 받기

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 3
(dy_dx,) = torch.autograd.grad(y, x, create_graph=True)
(d2y_dx2,) = torch.autograd.grad(dy_dx, x)
```

```
1번 미분 3x^2 : tensor(27., grad_fn=<MulBackward0>)
2번 미분 6x   : tensor(18.)
x.grad        : None
```

.grad에 쌓지 않고 결과로 돌려준다. create_graph=True면 기울기의 기울기도

입력에 대한 기울기: 무엇에 민감한가

판매량 = 2.5 × 기온 + 0.8 × 습도 + 0.05 × 기온 × 습도 − 15

기온 30°C, 습도 60%: 예측 **198.0개**
기온 1°C당 **5.50개** (2.5 + 0.05 × 60)
습도 1%p당 **2.30개** (0.8 + 0.05 × 30)

5.13 실습: 당뇨병 진행 정도를 autograd만으로

환자 442명 · 검사값 10가지 · 1년 뒤 진행 점수 → 350명 학습, 92명 시험 · 표준화 기준은 **학습 데이터에서만**

```
학습률, 배치_크기 = 0.05, 32
for 에폭 in range(1, 101):
    섞기 = torch.randperm(len(X_학습))
    for 시작 in range(0, len(X_학습), 배치_크기):
        고를 = 섞기[시작:시작 + 배치_크기]
        손실 = ((예측(X_학습[고를], w, b) - y_학습[고를]) ** 2).mean()
        손실.backward()
        with torch.no_grad():
            w -= 학습률 * w.grad
            b -= 학습률 * b.grad
        w.grad.zero_()
        b.grad.zero_()
```

```
기울기 일치: True True
에폭   1: 시험 RMSE  69.83
에폭   5: 시험 RMSE  52.52
에폭  20: 시험 RMSE  52.79
에폭 100: 시험 RMSE  52.25
```

평균만 찍으면 시험 RMSE **80.35**. 30점 가까이 덜 틀린다

5차시 · 5.13 실습 · 예제 05-autograd/b27_practice.py (학습 반복 부분, 출력에서 학습 RMSE 열은 뺐다)

5.13 정말 바닥일까: 공식으로 푼 정답과 비교



특성별 가중치 · ■ 최소제곱법 정답 · ■ 미니배치 100에폭 (앞 장)

전체 데이터로 5000번 걸은 경사하강법 vs 최소제곱법: 가장 큰 차이 **0.0004**, 시험 RMSE 52.35 = 52.35

그런데 s1·s2는 −43.18, 28.16 대 −25.44, 16.54. 시험 RMSE는 52.35와 52.25로 거의 같다

비슷하게 움직이는 두 검사값 → 평평한 긴 골짜기. 가중치 하나보다 **예측**을 보고 판단

5차시 · 5.13 실습(이어서) · 예제 05-autograd/b28_practice_exact.py · 그림은 b27·b28을 그대로 실행해 얻은 가중치로 그렸다

직접 실행 10분

1. b27_practice.py의 학습률을 0.01로, 다음엔 0.2로 바꾼다. 1에폭과 100에폭의 시험 RMSE는 어떻게 달라지나?
2. 배치_크기를 32에서 350(학습 데이터 전체)으로 바꾸면 5에폭과 100에폭의 시험 RMSE는?
3. b22_lr_regression.py의 학습률 목록을 [0.9, 1.0, 1.01]로 바꾼다. 어디서부터 망가지나?

```
cd examples/vol2/05-autograd && uv run python b27_practice.py
```

5차시 · 직접 실행 · b27은 2초 안쪽, b22는 1초 안쪽 · 씨앗이 고정돼 있어 모두 같은 숫자가 나온다

손실이 nan이 됐다. 가장 먼저 의심할 것은?

Q

가 배치 크기가 너무 작다

나 학습률이 너무 크다

다 에폭이 부족하다

라 no_grad를 너무 많이 썼다

5차시 정리

고칠 땐 **no_grad**, 예측만 할 땐 **inference_mode**, NumPy로 넘길 땐 **detach()**. 추적 중인 텐서엔 제자리 연산 금지

학습률은 너무 작으면 느리고 너무 크면 **발산(nan)**. 표준화해야 학습률 하나로 잘 내려간다

미니배치는 정확도와 걸음 수의 균형. backward()만으로 공식의 정답에 도착했다

다음: 2일차 6차시 · 이 일을 nn.Module과 옵티마이저에게 맡긴다

5차시 · 정리 · 복습: 책 5.8 ~ 5.14절

1일차에 배운 것

1. 텐서를 만들고 **모양을 이름 붙여 읽는다**. 슬라이스는 창
2. 브로드캐스팅·행렬 곱·dim 집계로 **한꺼번에** 계산한다
3. 기울기를 재고 **경사하강법**으로 바닥을 손으로 찾았다
4. **backward()**가 손으로 구한 기울기와 같은 값을 준다
5. 학습률·표준화·미니배치로 **실제 데이터**를 학습했다

내일 2일차 (6~10차시)

- 층, 활성화, nn.Module, 파라미터
- 손실, 옵티마이저, 표준 학습 루프
- 평가 루프, 저장, FashionMNIST 실습
- Dataset과 DataLoader
- CSV·이미지 폴더 Dataset, transforms

1일차 마무리 · 복습: 책 4장 전체, 5장 전체